Harness Amplification on SWE-bench Pro: An Evidence-First Case Study with OpenCodeOrchestra and Qwen3.6-27B

Aiden Kim

Technical report — May 2026

Abstract

How much can a harness amplify an accessible open-weight model on a public repository-repair benchmark? This report studies OpenCodeOrchestra (OCO), an opencode-based Project Manager → Orchestrator → specialist-agent workflow, on SWE-bench Pro with Qwen3.6-27B. In a seeded-subset pilot, OCO resolved 162/223 submitted patch rows: 72.6% strict (Wilson 95% CI 66.4–78.1%). Because the evaluation uses submitted rows from a seeded subset, the result is reported as an engineering pilot rather than a public leaderboard submission. The evidence suggests that harness design can materially change what an accessible model achieves. A later full-run deployment regression shows that harness, controller, and serving choices must be evaluated at task level, not only by throughput.

1 Introduction

A repository-level coding agent must inspect a codebase, interpret an issue, edit source, reason about tests, and submit a patch accepted by an external evaluator. The model matters, but it is only one component. The surrounding harness—prompt hierarchy, role decomposition, tools, permissions, review loops, retry policy, patch extraction, and model-serving stack—changes what the same model can accomplish.

This report studies harness amplification through OpenCodeOrchestra (OCO), a durable orchestration runtime for software-engineering tasks. The central question is not whether Qwen3.6-27B is a frontier model. It is how far a structured harness can push an accessible model on a public industrial benchmark, and what evidence is needed to study that effect responsibly.

The evidence record shows (i) a strong seeded-subset pilot under an archived B200 non-speculative profile, (ii) transparent denominator accounting, and (iii) a later deployment-profile regression caused by worse generated patches rather than evaluator failure. If the harness is the treatment, serving and controller changes are treatment changes as well.

The evidentiary standard used throughout is source-backed reporting. Load-bearing claims are tied to checked-in source, archived runner source, manifests, evaluator outputs, replay roots, and forensic notes.

Contributions.

1. A deterministic, repository-balanced seeded-subset pilot testing how far an OCO-style harness can push accessible Qwen3.6-27B on SWE-bench Pro.

2. A measured OCO harness configuration where PM-to-Orchestrator delegation was broadly observed and Auditor participation was available but not consistently enforced.

3. Validation of the positive patch bundle through replay on the later Modal/evaluator stack and manual inspection of representative passing histories.

4. A deployment-profile failure showing that harness changes must be judged by task outcomes, not only by token-throughput or patch-generation completeness.

2 Background and Related Work

Repository-level coding benchmarks. SWE-bench shifted coding-agent evaluation from short programming problems toward real GitHub issue resolution [4]. SWE-bench Verified introduced a human-filtered subset intended to reduce ambiguous tasks [9]. SWE-bench Pro extends the family toward longer-horizon, industrial repository tasks [2]. Task selection, evaluator version, bundle shape, and runtime budget are methodology variables, not incidental details.

Harness and agent-computer-interface effects. SWE-agent argued that the agent-computer interface is itself an experimental object: the same language model performs differently depending on observations, actions, shell/editor affordances, and submit loops [13]. mini-SWE-agent shows a compact scaffold can be strong and reproducible when the interface is clean [8]. Agentless shows that autonomous tool use is not automatically the winning variable; structured localization, repair, and validation can compete with more complex loops [12]. OpenHands provides a broad open platform for sandboxed software agents [11]. OCO belongs in this line of work: the harness is part of the treatment.

Multi-agent software-engineering systems. MetaGPT and ChatDev organized software work into teams of named roles [3, 7]. Repository-repair systems such as MAGIS, CodeR, and HyperAgent explore manager/developer/QA or task-graph decompositions [10, 1, 6]. OCO contributes a concrete runtime: persistent Orchestrator sessions, task-local specs, permissioned specialists, context-compression tooling, and audit-oriented workflow concepts built into a production fork of opencode.

Reviewer, verifier, and audit loops. Verifier and reviewer loops can improve code-generation systems when enforced and measured. OCO includes an Auditor concept; the positive bundle contains an audit-observed stratum. In this pilot, audit is a harness capability and observability variable; the measured treatment is the broader OCO runtime.

Harness and serving-profile sensitivity. Harness studies should treat serving as part of the runtime, not as a background detail. Quantization, KV-cache format, context length, batching, parser behavior, and speculative decoding can change more than throughput. A direct server-throughput A/B favored an H200 FP8 + MTP-4 profile, but downstream full-task generation quality regressed. Because B200-generated patches replayed successfully through the same evaluator while H200-generated patches failed by ordinary code-quality mechanisms, the case study motivates task-level quality A/B tests for the whole harness stack.

3 System Under Measurement: OpenCodeOrchestra

OCO extends opencode with a Project Manager → Orchestrator → specialist-agent workflow. PM gathers context and writes a task-local spec; Orchestrator owns implementation; Investigator and Auditor support evidence gathering and review. OCO source: `https://github.com/AidenGeunGeun/OpenCodeOrchestra` [5]. The benchmarked setup used OCO as an installed CLI; the benchmark repository does not modify OCO source.

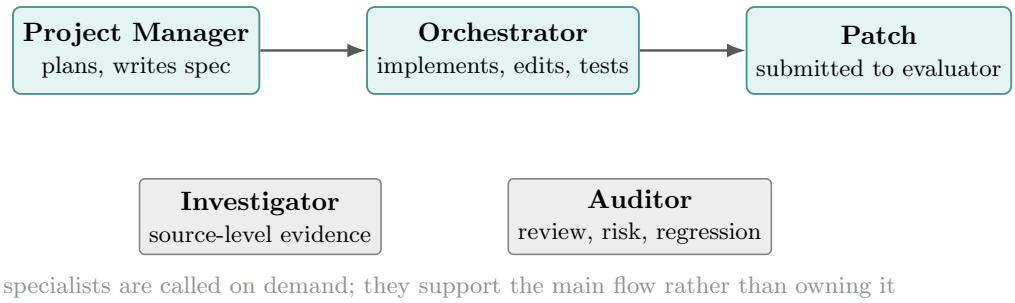

Figure 1: OCO workflow. PM owns task framing; Orchestrator owns implementation. Investigator and Auditor are support roles rather than extra implementation owners.

Component	Measured status in the positive pilot
PM → Orchestrator delegation	Broadly observed across the 223 clean rows; archived methodology reports all 223 had an observed Orchestrator task/session and 221 had delegation-like snippets.
Auditor loop	Available and prompted, not consistently enforced; archived split: 37 audit-observed, 185 audit-missing, 1 absence-justified.
Durable handoff back to PM	Often absent because resumed Orchestrator contexts frequently lacked the durable handoff tool; treated as telemetry caveat, not patch failure.
Direct PM edits	Common and recorded; representativeness warning for strict PM/Orchestrator separation.
Benchmark boundary	The benchmark consumes an installed OCO CLI and must not patch OCO source.

Table 1. What was actually measured in the positive pilot.

4 Experimental Protocol and Evidence Chain

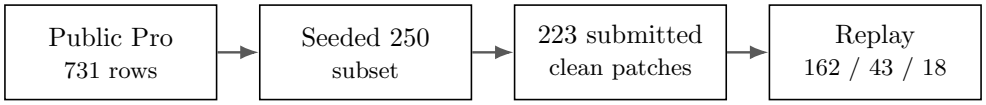

Figure 2: Evidence chain for the positive pilot. Each arrow is backed by an archived source artifact: controller pinning, seeded selection report, final bundle manifest, and current-evaluator replay outputs.

4.1 Task source and public split

The current controller pins SWE-bench Pro to `ScaleAI/SWE-bench_Pro`, split `test`, revision `7ab5114912baf22bb098818e604c02fe7ad2c11f`. It records the expected public split size of 731 rows, canonicalizes required fields, sorts by instance id, and writes SHA-256 manifests. This is not retroactive proof of the earlier pilot, but it defines the cleaned artifact standard now used by the repository.

4.2 Seeded 250-task subset

The positive pilot used a deterministic, repository-balanced subset. The archived runner grouped rows by repository, sorted rows within each repository by SHA-256 digest, sorted repositories by SHA-256 digest, then round-robined across repositories. The final seeded-250 pilot was the first 250 rows of a previously materialized full-731 order produced by that procedure. The archived preparation report records `selected_count=250`, `source_count=731`, selected IDs, per-repository counts, and the method string. The subset is repository-balanced but not category-balanced or deduplicated.

4.3 Positive-pilot serving profile

The archived positive run id is `qwen36-pro-full731-oco-b200-guarded-c5-20260520T050248Z`. The archived final methodology note identifies a Runpod B200 pod serving `Qwen/Qwen3.6-27B-FP8`, with 262,144-token server context, prefix caching, Qwen reasoning parser, non-streaming tool calls, and no speculative decoding. Metrics summaries record no speculative-acceptance counters.

Some later notes and replay-directory names call the positive bundle "NVFP4." This report follows the archived final methodology note for the quantization label and treats the mismatch as an artifact-labeling issue.

4.4 Evaluation path and replay

The final positive bundle contained 223 submitted clean patch rows and 223 matching raw rows out of the 250 selected tasks; the manifest records 27 excluded selected attempts with reasons. The bundle was later replayed through the current Modal/SWE-bench Pro evaluator checkout used for the H200 full-run evaluation. Parsing replay outputs gave **162 pass, 43 fail, 18 empty-test outputs** over 223 rows. The original archived combined tally was 158 / 46 / 19; the analysis uses 162/223 for current-evaluator comparability and discloses the historical archive tally alongside.

4.5 Denominator flow

Stage	Count	Evidence / interpretation
Seeded subset	250	Deterministic repository-balanced subset from the public 731-row order.
Submitted clean patch rows	223	Clean patches with matching raw rows in the final bundle manifest.
Excluded selected attempts	27	Infrastructure artifacts, invalid orchestration, empty/missing patches, dependency setup failure; not submitted.
Replay pass	162	Parsed as resolved in the current Modal/evaluator replay.
Replay fail	43	Meaningful evaluator non-pass among submitted rows.
Replay empty-test outputs	18	Manually inspected: mostly real non-resolutions, with a small evaluator/tooling-artifact subset.

Table 2. Denominator flow. The 223-row result measures submitted clean patches; the 250-row denominator is shown separately to avoid hiding non-submitted selected tasks.

5 Pilot Result

The positive pilot resolved 162 tasks in the current-evaluator replay of 223 submitted clean patch rows. Table 3 reports four denominators because they answer different questions: strict (primary evaluation-style number), artifact-adjusted (diagnostic validation-quality), known-only (optimistic diagnostic), and seeded-subset (conservative selected-task framing).

Framing	Count	Rate	Wilson 95% CI
Strict submitted-row	**162 / 223**	**72.6%**	**66.4–78.1%**
Artifact-adjusted diagnostic	162 / 219	74.0%	67.8–79.3%
Known-only diagnostic	162 / 205	79.0%	72.9–84.0%
Conservative seeded-subset	162 / 250	64.8%	58.7–70.5%

Table 3. Pilot result under transparent denominator choices. The strict submitted-row score is primary; adjusted scores are diagnostic.

Manual inspection of the 18 empty-test replay rows found no additional resolved tasks. Four were evaluator/tooling artifacts where no meaningful patch test occurred. Fourteen were real non-resolutions visible in build, import, runtime, or test-collection logs.

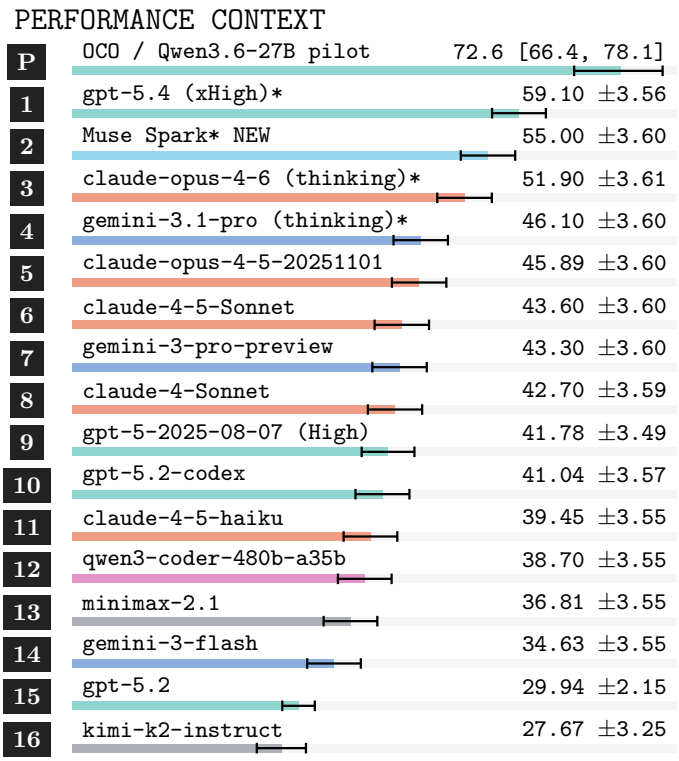

Figure 3: Published SWE-bench Pro performance context from Scale Labs values, with OCO's seeded-subset pilot added as a clearly labeled pilot row. The OCO row is not a leaderboard submission because it uses the seeded-subset submitted-patch denominator; it shows the magnitude of the engineering result against public full-benchmark scores. Whiskers show the reported uncertainty intervals.

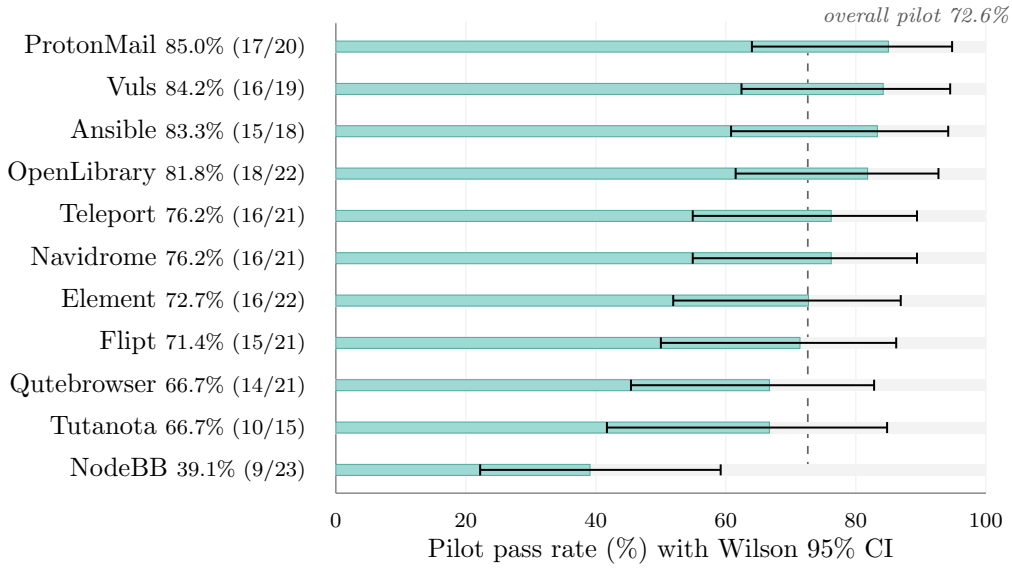

Figure 4: Per-repository pilot pass rate (162/223 submitted clean patches, current-evaluator replay), sorted by point estimate. Whiskers are Wilson 95% CIs; per-repo $n = 15$–23 makes intervals wide. NodeBB sits well below the rest of the field. The dashed line marks the overall pilot rate.

Representative passing trajectories. Two passing rows were inspected against archived OCO histories. A Navidrome task showed PM, Investigator, and Orchestrator activity; the patch fixed deterministic hashing behavior and replayed tests passed. An OpenLibrary task showed PM investigation, spec writing, Orchestrator delegation, vendor-language serialization edits, and 34 relevant replayed tests passing. These witnesses ground the result in task-relevant edits rather than evaluator or accounting artifacts.

6 Harness Sensitivity: Deployment-Profile Collapse

A later full 731-task generation run changed several variables at once: H200 hardware, FP8 served checkpoint, MTP-4 speculative decoding, and a new standalone controller path. It generated non-empty patches for all 731 public-split tasks. Official Modal evaluation produced **208 pass, 310 fail, 213 empty-test outputs**: 28.45% strict over the full split. Figure 5 shows the contrast directly.

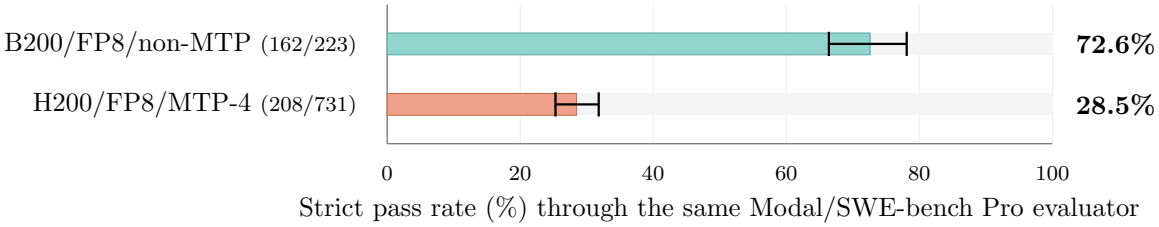

Figure 5: Same model family, same evaluator, two serving/controller profiles. Bars are strict pass rates; whiskers are Wilson 95% CIs. The intervals do not overlap: the difference is real, but the H200 run changed hardware, speculative decoding, and controller simultaneously, so it does not isolate a single causal variable.

The low H200 score initially raised the possibility of evaluator or bundle corruption. Three checks pointed back to generation quality: bundle validation found no duplicate, empty-patch, or raw/patch ID mismatch; raw-vs-sanitized evaluation behaved similarly on an unknown-heavy sample; and replaying the B200 patch bundle through the same current evaluator reproduced the high result.

Patch-level witnesses. Five tasks that passed in the B200 bundle and regressed in the H200 run were inspected patch-by-patch. The H200 patches failed by ordinary code-quality mechanisms: Flipt had incompatible config/exporter symbols; Navidrome left an expected helper undefined; Vuls missed macOS constants and scanner wiring; Teleport introduced a different matcher API; and OpenLibrary changed language serialization into the wrong representation. The H200 FP8 + MTP-4 run is therefore treated as a deployment-profile failure. For harness research, the implication is that changes which make an agent faster, cheaper, or easier to run still require task-level validation.

7 Artifact Availability and Reproducibility

The repository contains controller code, run notes, result summaries, forensic reports, and this paper. Large bundles and attempt histories are excluded from Git because they contain hundreds of megabytes to many gigabytes of patches, evaluator outputs, and worktrees. The evidence record has three classes:

- **Checked-in source evidence:** current controller task-source pinning and canonicalization; documentation of result policy and H200 run deviations.
- **Archived local evidence:** runner source, seeded-250 preparation report, final-223 bundle manifest, attempt histories, and original evaluation outputs under the external archive.
- **Pod/replay evidence:** current Modal replay output roots and H200 raw/sanitized bundle records, including SHA-256 hashes and sanitizer removal manifests.

A compact public artifact should include selected instance IDs, per-row replay status, excluded artifact-row IDs and reasons, bundle hashes, and witness summaries. This would allow readers to audit the methodology without downloading full worktrees.

Known artifact-label caveat. Legacy filenames and some later notes refer to the positive replay bundle as "old NVFP4." The archived final B200 seeded-250 methodology note records `Qwen/Qwen3.6-27B-FP8`. This report follows the archived methodology note. Public artifacts should either recover the original Runpod serving command or rename legacy replay labels so the quantization description is unambiguous.

8 Threats to Validity

No matched baseline. The positive pilot lacks a same-model, same-serving mini-SWE-agent or SWE-agent baseline on the exact seeded subset. It is strongest as an engineering pilot and artifact-backed case study. Matched baselines and serving-profile A/B tests are required to estimate the effect size of OCO relative to alternative harnesses.

Subset and non-submission. The 250-task subset was deterministic and repository-balanced, but it was still a subset. Only 223 selected rows became submitted clean patch rows. The strict 223-row score measures submitted patch quality; the 250-row conservative score exposes the cost of non-submission.

Evaluator artifacts and manual classification. Manual classification of empty-test outputs can introduce judgment. This is why the strict 162/223 number stays primary and the 162/219 artifact-adjusted score is labeled diagnostic.

Audit loop not enforced. OCO's Auditor was available but most clean patches did not show an observed Auditor pass. Audit is a runtime feature in this report, not the measured treatment.

Harness ablations. A controlled ablation study is needed to isolate the contribution of each role. Relevant conditions include Orchestrator-only repair; PM → Orchestrator without Auditor enforcement; PM → Orchestrator with a required Auditor pass; and PM-led edit with Auditor review, which models the direct-edit paths observed in this pilot. Such a design would estimate which harness components account for the observed gain.

Serving/controller confounding. The H200 collapse changed hardware, quantization/checkpoint, MTP, controller behavior, prompt/tool surface, and continuation policy. Replay and witness forensics isolate the collapse away from evaluator-only explanations but do not identify a single causal variable.

Agent-assisted benchmark implementation. The benchmark harness and documentation were developed with agent assistance. This increases the need for source-backed claims, manifests, hashes, and independent re-runs before any formal publication.

Artifact access. Some evidence currently lives in local external storage or pod-local paths. Full independent reproduction requires compact public manifests and shareable replay artifacts.

9 Conclusion

OCO's seeded-subset pilot shows that a durable orchestration harness can produce strong repository-repair results with accessible Qwen3.6-27B under an archived B200 non-speculative profile. The larger research question is harness amplification: how much performance comes from the model, and how much comes from the runtime surrounding it? The H200 full-run regression makes the systems lesson sharper: harness design, serving profile, and evaluator evidence are first-class experimental variables in coding-agent research.

References

[1] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue resolving with multi-agent and task graphs, 2024. Repository: `https://github.com/NL2Code/CodeR`.

[2] Xiang Deng, Jeff Da, Edwin Pan, Yan He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean M. Hendryx, Zifan Wang, et al. SWE-bench pro: Can AI agents solve long-horizon software engineering tasks?, 2025. Project repository: `https://github.com/scaleapi/SWE-bench_Pro-os`.

[3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. OpenReview: `https://openreview.net/forum?id=VtmBAGCN7o`.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. OpenReview: `https://openreview.net/forum?id=VTF8yNQM66`.

[5] Aiden Kim. OpenCodeOrchestra, 2026. Source repository: `https://github.com/AidenGeunGeun/OpenCodeOrchestra`.

[6] Huy Nhat Phan, Phong X. Nguyen, and Nghi D. Q. Bui. HyperAgent: Generalist software engineering agents to solve coding tasks at scale, 2024. arXiv/project metadata vary across public pages; verify exact arXiv identifier before formal submission. Repository: `https://github.com/FSoft-AI4Code/HyperAgent`.

[7] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. ACL Anthology: `https://aclanthology.org/2024.acl-long.810/`.

[8] SWE-agent Team. mini-SWE-agent, 2025. Project repository: `https://github.com/SWE-agent/mini-swe-agent`.

[9] SWE-bench Team and OpenAI. SWE-bench verified, 2024. Human-filtered 500-task subset. `https://www.swebench.com/verified`; announcement: `https://openai.com/index/introducing-swe-bench-verified/`.

[10] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems*, 2024. OpenReview: `https://openreview.net/forum?id=qevq3FZ63J`.

[11] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. OpenReview: `https://openreview.net/forum?id=OJd3ayDDoF`; repository: `https://github.com/OpenHands/OpenHands`.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025. Preprint: `https://arxiv.org/abs/2407.01489`.

[13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. arXiv: `https://arxiv.org/abs/2405.15793`.